



Smart Contract Security Audit Report



Table Of Contents

1 Executive Summary	_____
2 Audit Methodology	_____
3 Project Overview	_____
3.1 Project Introduction	_____
3.2 Vulnerability Information	_____
4 Code Overview	_____
4.1 Contracts Description	_____
4.2 Visibility Description	_____
4.3 Vulnerability Summary	_____
5 Audit Result	_____
6 Statement	_____

1 Executive Summary

On 2026.03.18, the SlowMist security team received the Multipli team's security audit application for rwaUSD, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.

Level	Description
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit

Serial Number	Audit Class	Audit Subclass
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

This is multipli's RWAUSD protocol, which mainly includes dss and token modules.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Missing payable fallback function	Others	Suggestion	Fixed
N2	Chainlink answeredInRound is deprecated	Design Logic Audit	Suggestion	Fixed
N3	L2 Sequencer Offline Risk	Design Logic Audit	Suggestion	Fixed
N4	Single Point of Failure Risk	Authority Control Vulnerability Audit	Suggestion	Acknowledged
N5	Missing decimals Validation	Integer Overflow and Underflow Vulnerability	Medium	Fixed
N6	Hope/Nope event parameters are reversed	Design Logic Audit	Low	Fixed
N7	Missing Event Logs	Others	Suggestion	Fixed
N8	Proxy deployment may bind to the wrong Vat	Others	Suggestion	Fixed
N9	Lack of Consistency Validation for Core System Addresses	Others	Suggestion	Fixed
N10	Risk of Excessive Privilege	Authority Control Vulnerability Audit	Low	Acknowledged

NO	Title	Category	Level	Status
N11	Missing zero address check	Others	Suggestion	Fixed
N12	Repeatedly triggered events	Others	Suggestion	Fixed
N13	ERC-7201 Storage slot namespace mismatch	Others	Medium	Fixed

4 Code Overview

4.1 Contracts Description

Part 1:

<https://github.com/brine-finance/rwausd-dss>

Initial audit version: aa44bed39ff4ee6f41f0a50c356b992650942997

Final audit version: 7715ba8782fb879bed21c6322096fa04bd02c6db

Part 2:

<https://github.com/brine-finance/rwausd-token-ccip>

Initial audit version: e877131f8cdc2b068dc9acb423f37d3da6ee44ed

Final audit version: ae89c53fe809ee81959e9f8d2aff61ce4a2d9a5b

Audit Scope:

```

rwausd-dss
.
├── script
│   ├── collateral
│   │   └── build
│   └── AddBuildOnAnvil.s.sol

```

```

├── AddBuidlOnEthereumMainnet.s.sol
├── AddBuidlOnEthereumTestnet.s.sol
├── common
│   ├── BaseAddCollateral.s.sol
│   ├── BaseDeployProxies.s.sol
│   └── BaseDeploySystem.s.sol
├── protocol
│   ├── DeploySystemOnAnvil.s.sol
│   ├── DeploySystemOnEthereumMainnet.s.sol
│   └── DeploySystemOnEthereumTestnet.s.sol
├── proxy
│   ├── DeployProxiesOnAnvil.s.sol
│   ├── DeployProxiesOnEthereumMainnet.s.sol
│   └── DeployProxiesOnEthereumTestnet.s.sol
└── src
    ├── abaci.sol
    ├── adapter.sol
    ├── clip.sol
    ├── cure.sol
    ├── dog.sol
    ├── end.sol
    ├── flap.sol
    ├── flop.sol
    ├── join.sol
    ├── jug.sol
    ├── osm.sol
    ├── proxy
    │   ├── DssCdpManager.sol
    │   ├── DssProxy.sol
    │   ├── DssProxyActions.sol
    │   ├── GetCdps.sol
    │   ├── ProxyRegistry.sol
    │   └── proxy.sol
    ├── spot.sol
    ├── vat.sol
    └── vow.sol

```

```
rwausd-token-ccip
```

```
./src
```

```
|— interfaces
|   └─ IBurnMintERC20.sol
|— token
|   └─ RwaUsd.sol
```



```
└─ upgradeable
    └─ BurnMintERC20Upgradeable.sol
```

The main network address of the contract is as follows:

rwaUSD Token

0x8Fcd23142047A3073ed332a0Ed07d1e8D2BD5177

Vat

0xbC22e8C15bC476EF4FD0124c5A03b23607e30D2C

Spotter

0xf3aee748355bb07CBe702B4ff8dBE6118b34e2A2

rwaUSDJoin

0xA596E0888b3e1d12F3Ee174128c0D9990027BA08

Jug

0x66654ccF7C4492FD3CFa01A7Ec5B5AC28f0D886E

Vow

0x7815e8e9BCEF8708A799eE3802586298e5AFA611

Dog

0x15a36d5cAf263160c2a49DDE6429C045Fb711dDD

Cure

0x9eA54efa0D82B2afdf2c9F3A407622Dc83b2c914

End

0x026782F431bfC233c67128af42a4e9De7f834BF5

DssCdpManager

0x97a762D03a511754151dD046EE83599162572E47

DssProxyActions

0x92a4bb65fBe7A4b516F52A0e7550982f650Ce7c8

DssProxyActionsEnd

0x41c09d97E10dDB1AdbC202D6fC3AA18547c36969

DSPProxyFactory

0xE23c0D47cDcb3dF144c529648651D5C8F3cBe312

ProxyRegistry

0xAfd44c3e4BB4757527D1d58706023DAf2f1f30D3

GetCdps

0x2FCC0d9972f489Ec42804a00bA2029E7f6C9b412

PriceFeedAdapter

0x82f5790BD1c96790E4c3A3EBc8142bd4D6f8B1cd

OSM

0x89fbae0302b8790d55fA36e6aB09aC93f865993a

GemJoin5

0x3C9567c3b9C20e72858cD5714209EA7D7a8011fd

StairstepExponentialDecrease

0xFEb42fb58E790dD5f38D936DF45b4bDD29260De5

Clipper

0x62b7a353928142a18c07026a33f8089D1c7378f4

MULTIPLI_SAFE_WALLET

0x194Ebc1B9B382ef0E6998cAAcE59aF843cf53b99

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

rwaUSD			
Function Name	Visibility	Mutability	Modifiers
_getRwaUsdStorage	Private	-	-
<Constructor>	Public	Can Modify State	-
initialize	Public	Can Modify State	initializer
__RwaUsd_init	Internal	Can Modify State	onlyInitializing
__RwaUsd_init_unchained	Internal	Can Modify State	onlyInitializing
_authorizeUpgrade	Internal	Can Modify State	onlyRole
supportsInterface	Public	-	-
decimals	Public	-	-
maxSupply	Public	-	-
burn	Public	Can Modify State	onlyRole
burn	Public	Can Modify State	-
burnFrom	Public	Can Modify State	onlyRole
mint	External	Can Modify State	onlyRole
grantMintAndBurnRoles	External	Can Modify State	-
getCCIPAdmin	External	-	-
setCCIPAdmin	External	Can Modify State	onlyRole
pause	Public	Can Modify State	onlyRole

rwaUSD			
unpause	Public	Can Modify State	onlyRole
_update	Internal	Can Modify State	-
_approve	Internal	Can Modify State	-

LinearDecrease			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
file	External	Can Modify State	auth
add	Internal	-	-
mul	Internal	-	-
rmul	Internal	-	-
price	External	-	-

StairstepExponentialDecrease			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
file	External	Can Modify State	auth

StairstepExponentialDecrease			
rmul	Internal	-	-
rpow	Internal	-	-
price	External	-	-

ExponentialDecrease			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
file	External	Can Modify State	auth
rmul	Internal	-	-
rpow	Internal	-	-
price	External	-	-

PriceFeedAdapter			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
transferOwnership	External	Can Modify State	onlyOwner
setAsset	External	Can Modify State	onlyOwner
setPriceFeed	External	Can Modify State	onlyOwner
setMaxDelay	External	Can Modify State	onlyOwner

PriceFeedAdapter			
pause	External	Can Modify State	onlyOwner
unpause	External	Can Modify State	onlyOwner
peek	Public	-	-
read	External	-	-

Clipper			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
file	External	Can Modify State	auth lock
file	External	Can Modify State	auth lock
min	Internal	-	-
add	Internal	-	-
sub	Internal	-	-
mul	Internal	-	-
wmul	Internal	-	-
rmul	Internal	-	-
rdiv	Internal	-	-
getFeedPrice	Internal	Can Modify State	-
kick	External	Can Modify State	auth lock isStopped

Clipper			
redo	External	Can Modify State	lock isStopped
take	External	Can Modify State	lock isStopped
_remove	Internal	Can Modify State	-
count	External	-	-
list	External	-	-
getStatus	External	-	-
status	Internal	-	-
upchost	External	Can Modify State	-
yank	External	Can Modify State	auth lock

Cure			
Function Name	Visibility	Mutability	Modifiers
_add	Internal	-	-
_sub	Internal	-	-
<Constructor>	Public	Can Modify State	-
tCount	External	-	-
list	External	-	-
tell	External	-	-
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth

Cure			
file	External	Can Modify State	auth
lift	External	Can Modify State	auth
drop	External	Can Modify State	auth
cage	External	Can Modify State	auth
load	External	Can Modify State	-

Dog			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
min	Internal	-	-
add	Internal	-	-
sub	Internal	-	-
mul	Internal	-	-
file	External	Can Modify State	auth
file	External	Can Modify State	auth
file	External	Can Modify State	auth
file	External	Can Modify State	auth
chop	External	-	-
bark	External	Can Modify State	-

Dog			
digs	External	Can Modify State	auth
cage	External	Can Modify State	auth

End			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
add	Internal	-	-
sub	Internal	-	-
mul	Internal	-	-
min	Internal	-	-
rmul	Internal	-	-
wdiv	Internal	-	-
file	External	Can Modify State	auth
file	External	Can Modify State	auth
cage	External	Can Modify State	auth
cage	External	Can Modify State	-
snip	External	Can Modify State	-
skim	External	Can Modify State	-

End			
free	External	Can Modify State	-
thaw	External	Can Modify State	-
flow	External	Can Modify State	-
pack	External	Can Modify State	-
cash	External	Can Modify State	-

Flapper			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
add	Internal	-	-
add256	Internal	-	-
sub	Internal	-	-
mul	Internal	-	-
file	External	Can Modify State	auth
kick	External	Can Modify State	auth
tick	External	Can Modify State	-
tend	External	Can Modify State	-
deal	External	Can Modify State	-
cage	External	Can Modify State	auth

Flapper			
yank	External	Can Modify State	-

Flopper			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
add	Internal	-	-
mul	Internal	-	-
min	Internal	-	-
file	External	Can Modify State	auth
kick	External	Can Modify State	auth
tick	External	Can Modify State	-
dent	External	Can Modify State	-
deal	External	Can Modify State	-
cage	External	Can Modify State	auth
yank	External	Can Modify State	-

GemJoin5			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth

GemJoin5			
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
cage	External	Can Modify State	auth
mul	Internal	-	-
join	Public	Can Modify State	-
exit	Public	Can Modify State	-

rwaUSDJoin			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
cage	External	Can Modify State	auth
mul	Internal	-	-
join	External	Can Modify State	-
exit	External	Can Modify State	-

Jug			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth

Jug			
<Constructor>	Public	Can Modify State	-
rpow	Internal	-	-
add	Internal	-	-
diff	Internal	-	-
rmul	Internal	-	-
init	External	Can Modify State	auth
file	External	Can Modify State	auth
file	External	Can Modify State	auth
file	External	Can Modify State	auth
drip	External	Can Modify State	-

OSM			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
add	Internal	-	-
<Constructor>	Public	Can Modify State	-
stop	External	Can Modify State	auth
start	External	Can Modify State	auth
change	External	Can Modify State	auth
era	Internal	-	-

OSM			
prev	Internal	-	-
step	External	Can Modify State	auth
void	External	Can Modify State	auth
pass	Public	-	-
poke	External	Can Modify State	stoppable
peek	External	-	toll
peep	External	-	toll
read	External	-	toll
kiss	External	Can Modify State	auth
diss	External	Can Modify State	auth
kiss	External	Can Modify State	auth
diss	External	Can Modify State	auth

Spotter			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-
mul	Internal	-	-
rdiv	Internal	-	-

Spotter			
file	External	Can Modify State	auth
file	External	Can Modify State	auth
file	External	Can Modify State	auth
poke	External	Can Modify State	-
cage	External	Can Modify State	auth

Vat			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
hope	External	Can Modify State	-
nope	External	Can Modify State	-
wish	Internal	-	-
<Constructor>	Public	Can Modify State	-
add	Internal	-	-
sub	Internal	-	-
mul	Internal	-	-
add	Internal	-	-
sub	Internal	-	-
mul	Internal	-	-
init	External	Can Modify State	auth

Vat			
file	External	Can Modify State	auth
file	External	Can Modify State	auth
cage	External	Can Modify State	auth
slip	External	Can Modify State	auth
flux	External	Can Modify State	-
move	External	Can Modify State	-
either	Internal	-	-
both	Internal	-	-
frob	External	Can Modify State	-
fork	External	Can Modify State	-
grab	External	Can Modify State	auth
heal	External	Can Modify State	-
suck	External	Can Modify State	auth
fold	External	Can Modify State	auth

Vow			
Function Name	Visibility	Mutability	Modifiers
rely	External	Can Modify State	auth
deny	External	Can Modify State	auth
<Constructor>	Public	Can Modify State	-

Vow			
add	Internal	-	-
sub	Internal	-	-
min	Internal	-	-
file	External	Can Modify State	auth
file	External	Can Modify State	auth
fess	External	Can Modify State	auth
flog	External	Can Modify State	-
heal	External	Can Modify State	-
kiss	External	Can Modify State	governanceEnabled
flop	External	Can Modify State	governanceEnabled
flap	External	Can Modify State	governanceEnabled
cage	External	Can Modify State	auth

UrnHandler			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-

DssCdpManager			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
add	Internal	-	-

DssCdpManager			
sub	Internal	-	-
tolnt	Internal	-	-
cdpAllow	Public	Can Modify State	cdpAllowed
urnAllow	Public	Can Modify State	-
open	Public	Can Modify State	-
give	Public	Can Modify State	cdpAllowed
frob	Public	Can Modify State	cdpAllowed
flux	Public	Can Modify State	cdpAllowed
flux	Public	Can Modify State	cdpAllowed
move	Public	Can Modify State	cdpAllowed
quit	Public	Can Modify State	cdpAllowed urnAllowed
enter	Public	Can Modify State	urnAllowed cdpAllowed
shift	Public	Can Modify State	cdpAllowed cdpAllowed

DssProxy			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
<Receive Ether>	External	Payable	-
setOwner	External	Can Modify State	auth
setAuthority	External	Can Modify State	auth
execute	External	Payable	auth

DssProxy			

Common			
Function Name	Visibility	Mutability	Modifiers
mul	Internal	-	-
rwaUSDJoin_join	Public	Can Modify State	-

DssProxyActions			
Function Name	Visibility	Mutability	Modifiers
sub	Internal	-	-
toInt	Internal	-	-
toRad	Internal	-	-
convertTo18	Internal	Can Modify State	-
_getDrawDart	Internal	Can Modify State	-
_getWipeDart	Internal	-	-
_getWipeWad	Internal	-	-
_getWipeAllWad	Internal	-	-
transfer	Public	Can Modify State	-
gemJoin_join	Public	Can Modify State	-
hope	Public	Can Modify State	-
nope	Public	Can Modify State	-
open	Public	Can Modify State	-

DssProxyActions			
give	Public	Can Modify State	-
giveToProxy	Public	Can Modify State	-
cdpAllow	Public	Can Modify State	-
urnAllow	Public	Can Modify State	-
flux	Public	Can Modify State	-
move	Public	Can Modify State	-
frob	Public	Can Modify State	-
quit	Public	Can Modify State	-
enter	Public	Can Modify State	-
shift	Public	Can Modify State	-
makeGemBag	Public	Can Modify State	-
lockGem	Public	Can Modify State	-
safeLockGem	Public	Can Modify State	-
freeGem	Public	Can Modify State	-
exitGem	Public	Can Modify State	-
draw	Public	Can Modify State	-
wipe	Public	Can Modify State	-
safeWipe	Public	Can Modify State	-
wipeAll	Public	Can Modify State	-
safeWipeAll	Public	Can Modify State	-

DssProxyActions			
lockGemAndDraw	Public	Can Modify State	-
openLockGemAndDraw	Public	Can Modify State	-
wipeAndFreeGem	Public	Can Modify State	-
wipeAllAndFreeGem	Public	Can Modify State	-

DssProxyActionsEnd			
Function Name	Visibility	Mutability	Modifiers
_free	Internal	Can Modify State	-
freeGem	Public	Can Modify State	-
pack	Public	Can Modify State	-
cashGem	Public	Can Modify State	-

GetCdps			
Function Name	Visibility	Mutability	Modifiers
getCdpsAsc	External	-	-
getCdpsDesc	External	-	-

DSPProxy			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
<Receive Ether>	External	Payable	-
execute	Public	Payable	-

DSProxy			
execute	Public	Payable	auth
setCache	Public	Can Modify State	auth

DSProxyFactory			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
build	Public	Can Modify State	-
build	Public	Can Modify State	-

DSProxyCache			
Function Name	Visibility	Mutability	Modifiers
read	Public	-	-
write	Public	Can Modify State	-

ProxyRegistry			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
build	Public	Can Modify State	-
build	Public	Can Modify State	-

4.3 Vulnerability Summary

[N1] [Suggestion] Missing payable fallback function

Category: Others**Content**

The upstream code used `function() external payable`, which accepts "any unmatched call," including calls "with ETH and non-empty calldata." However, after the upgrade, only "ETH transfers with empty calldata" will enter `receive()`. Unmatched calls with non-empty calldata will now revert directly.

- rwausd-dss/src/proxy/proxy.sol#L34

```
receive() external payable {}
```

Solution

If the old behavior needs to be maintained, it is recommended to add `fallback() external payable {}`.

Status

Fixed

[N2] [Suggestion] Chainlink answeredInRound is deprecated**Category: Design Logic Audit****Content**

Chainlink has officially confirmed that `answeredInRound` is a deprecated field. In modern OCR aggregators, `answeredInRound` is generally equal to `roundId`, causing this check to practically never trigger.

- rwaUSD/rwausd-dss/src/adapter.sol

```
function peek() public view override returns (bytes32 val, bool has) {
    // ...
    // answeredInRound < roundId means the answer is from a stale round
    if (answeredInRound < roundId) return (bytes32(0), false);
    // ...
}
```

Solution

It is recommended to remove this check.

Status

Fixed

[N3] [Suggestion] L2 Sequencer Offline Risk**Category: Design Logic Audit****Content**

If the contract is deployed on L2 networks like Arbitrum / Optimism, the price will stagnate while the Sequencer is offline. In the instant it recovers, there will be an MEV arbitrage window.

- rwausd-dss/src/adapters.sol#L102-L125

```
function peek() public view override returns (bytes32 val, bool has) {  
    // ...  
}
```

Solution

It is recommended to integrate the Chainlink L2 Sequencer Uptime Feed and add a Sequencer status check in

`peek()`.

Status

Fixed

[N4] [Suggestion] Single Point of Failure Risk**Category: Authority Control Vulnerability Audit****Content**

All critical operations are controlled by a single owner, with no timelock, no multisig, and no secondary confirmation.

If the private key is compromised, an attacker can immediately inject a malicious Feed and manipulate the price across the entire system.

- rwausd-dss/src/adapter.sol#L62-L66, L69-L73, L75-L79, L81-L85, L88-L91, L93-L96

```
function transferOwnership(address newOwner) external onlyOwner {}

function setAsset(address newAsset) external onlyOwner {}

function setPriceFeed(address newFeed) external onlyOwner {}

function setMaxDelay(uint256 newDelay) external onlyOwner {}

function pause() external onlyOwner {}

function unpause() external onlyOwner {}
```

Solution

It is recommended to implement role separation, assigning different operations to different roles; implement a two-step ownership transfer; and add a Timelock for modifications to critical parameters.

Status

Acknowledged

[N5] [Medium] Missing decimals Validation

Category: Integer Overflow and Underflow Vulnerability

Content

Currently, `GemJoin5` does not restrict the collateral token's decimals to be less than or equal to 18. Therefore, if a token with decimals > 18 is integrated, the `10 ** (18 - dec)` used for precision conversion in `join` and `exit` will cause an unsigned subtraction underflow in Solidity 0.6.12. This will generate an incorrect scaling factor, resulting in severe distortion between the internal Vat accounting and the actual tokens transferred in/out.

- rwausd-dss/src/join.sol#L67-L74

```
constructor(address vat_, bytes32 ilk_, address gem_) public {
    gem = GemLike5(gem_);
    dec = gem.decimals();
```

```
wards[msg.sender] = 1;
live = 1;
vat = VatLike(vat_);
ilk = ilk_;
}
```

Solution

Explicitly validate `dec <= 18` in the `GemJoin5` constructor to avoid precision conversion underflows and subsequent accounting errors.

Status

Fixed

[N6] [Low] Hope/Nope event parameters are reversed

Category: Design Logic Audit

Content

The Event is defined as `Hope(u, v)`, but it is emitted as `(usr, msg.sender)`, meaning `u = usr` (the authorized party) and `v = msg.sender` (the authorizing party). Semantically, `u` usually refers to the initiator (authorizing party) and `v` refers to the target (authorized party). The parameter order is opposite to the semantic meaning.

- rwausd-dss/src/vat.sol#L74-L77, L79-L82

```
function hope(address usr) external {
    can[msg.sender][usr] = 1;
    emit Hope(usr, msg.sender);
}

function nope(address usr) external {
    can[msg.sender][usr] = 0;
    emit Nope(usr, msg.sender);
}
```

Solution

It is recommended to uniformly emit `Hope(msg.sender, usr)`.

Status

Fixed

[N7] [Suggestion] Missing Event Logs

Category: Others

Content

In the Vow contract, `cage()` did not trigger the `Cage()` event.

- rwausd-dss/src/vow.sol#L207-L217

```
function cage() external auth {
    require(live == 1, "Vow/not-live");
    live = 0;
    Sin = 0;
    Ash = 0;
    if (address(flopper) != address(0) && address(flapper) != address(0)) {
        flapper.cage(vat.rwaUSD(address(flapper)));
        flopper.cage();
    }
    vat.heal(min(vat.rwaUSD(address(this)), vat.sin(address(this))));
}
```

Solution

It is recommended to add event logging.

Status

Fixed

[N8] [Suggestion] Proxy deployment may bind to the wrong Vat

Category: Others

Content

The current Proxy deployment process only checks that the `VAT` is not zero. It does not verify whether it is consistent with the actual `Vat` used by the online core system. Therefore, the deployment process may appear to

complete successfully, but the `DssCdpManager` is actually bound to an incorrect ledger instance, creating a potential system consistency risk that may only be exposed during subsequent `CDP` operations.

- rwausd-dss/script/common/BaseDeployProxies.s.sol#L43-L62

```
function run() public broadcastOrPrank {
    setDeploymentConfig();
    require(VAT != address(0), "BaseDeployCdpAndProxy: VAT not set");
    // ...
}
```

Solution

It is recommended that the Proxy deployment process directly use the Vat address from the core system's standard deployment artifacts, or perform a consistency check between the provided VAT and the Vat address registered in the core system before deployment; if they are inconsistent, the deployment should be aborted immediately.

Status

Fixed

[N9] [Suggestion] Lack of Consistency Validation for Core System Addresses

Category: Others

Content

The current `AddCollateral` process only verifies that the core system addresses are not zero, rather than verifying whether these addresses belong to the same valid online core system. Therefore, even if passed non-zero addresses that are expired, incorrect, or mismatched with one another, the script may continue executing and configure the wrong targets. This leads to collateral being connected to inconsistent system components, introducing high configuration and operational risks.

- rwausd-dss/script/common/BaseAddCollateral.s.sol#L78-L90

```
function validateConfig() internal virtual {
    require(OWNER == msg.sender, "OWNER is not the deployer");
```

```
require(VAT != address(0), "VAT not set");
require(SPOTTER != address(0), "SPOTTER not set");
require(DOG != address(0), "DOG not set");
require(JUG != address(0), "JUG not set");
require(VOW != address(0), "VOW not set");
require(END != address(0), "END not set");
require(TOKEN != address(0), "TOKEN not set");
require(ORACLE != address(0), "ORACLE not set");
require(ILK != bytes32(0), "ILK not set");
require(ILK_LINE > 0, "ILK_LINE not set");
}
```

Solution

It is recommended to add a core address consistency check before executing `AddCollateral`. This check should not only verify that the address is not zero, but also verify whether the key relationships between the core components are valid.

Status

Fixed

[N10] [Low] Risk of Excessive Privilege

Category: Authority Control Vulnerability Audit

Content

1. `DSS wards` are the highest-privileged administrators of the core protocol contracts. They can add or remove other administrators via `rely/deny`, and can use privileged entry points such as `file` and `cage` to modify critical system parameters, adjust liquidation/interest-rate/debt-ceiling configurations, or trigger emergency shutdown, effectively giving them full control over protocol operations.

- `rwausd-dss/src/vat.sol`#L56-L60, L62-L66, L151-L155, L157-L162, L164-L171, L173-L176

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function init(bytes32 ilk) external auth {}
```

```
function file(bytes32 what, uint256 data) external auth {}

function file(bytes32 ilk, bytes32 what, uint256 data) external auth {}

function cage() external auth {}
```

- rwausd-dss/src/spot.sol#L40-L43, L45-L48, L90-L95, L97-L102, L104-L109, L119-L122

```
function rely(address guy) external auth {}

function deny(address guy) external auth {}

function file(bytes32 ilk, bytes32 what, address pip_) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function file(bytes32 ilk, bytes32 what, uint256 data) external auth {}

function cage() external auth {}
```

- rwausd-dss/src/jug.sol#L47-L50, L52-L55, L128-L134, L136-L141, L143-L147, L149-L153

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function init(bytes32 ilk) external auth {}

function file(bytes32 ilk, bytes32 what, uint256 data) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function file(bytes32 what, address data) external auth {}
```

- rwausd-dss/src/join.sol#L47-L50, L52-L55, L76-L78, L113-L116, L118-L121, L138-L141

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}
```

```
function cage() external auth {}

function rely(address usr) external auth {}

function deny(address usr) external auth {}

function cage() external auth {}
```

- rwausd-dss/src/flop.sol#L55-L57, L59-L61, L116-L122, L179-L182

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function cage() external auth {}
```

- rwausd-dss/src/flap.sol#L50-L52, L54-L56, L114-L120, L176-L179

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function cage(uint256 rad) external auth {}
```

- rwausd-dss/src/end.sol#L207-L210, L212-L215, L296-L305, L307-L312, L315-L325

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, address data) external auth {}

function file(bytes32 what, uint256 data) external auth {}
```

```
function cage() external auth {}
```

- rwausd-dss/src/dog.sol#L58-L61, L63-L66, L138-L142, L144-L148, L150-L160, L162-L170, L256-L259

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, address data) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function file(bytes32 ilk, bytes32 what, uint256 data) external auth {}

function file(bytes32 ilk, bytes32 what, address clip) external auth {}

function cage() external auth {}
```

- rwausd-dss/src/cure.sol#L82-L86, L88-L92, L94-L99, L101-L107, L109-L123, L125-L130

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function lift(address src) external auth {}

function drop(address src) external auth {}

function cage() external auth {}
```

- rwausd-dss/src/clip.sol#L55-L58, L60-L63, L170-L179, L181-L188, L496-L502

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth lock {}
```



```
function file(bytes32 what, address data) external auth lock {}

function yank(uint256 id) external auth lock {}
```

- rwausd-dss/src/osm.sol#L26-L29, L31-L34, L91-L94, L96-L99, L101-L104, L115-L119, L121-L125, L155-L159, L161-L164, L166-L172, L174-L179

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function stop() external auth {}

function start() external auth {}

function change(address src_) external auth {}

function step(uint16 ts) external auth {}

function void() external auth {}

function kiss(address a) external auth {}

function diss(address a) external auth {}

function kiss(address[] calldata a) external auth {}

function diss(address[] calldata a) external auth {}
```

- rwausd-dss/src/vow.sol#L76-L80, L82-L85, L139-L148, L150-L161, L207-L217

```
function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function file(bytes32 what, address data) external auth {}

function cage() external auth {}
```

- rwausd-dss/src/abaci.sol#L31-L34, L36-L39, L61-L65, L103-L106, L108-L111, L136-L141, L206-L209, L211-L214, L238-L242

```
function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth {}

function rely(address usr) external auth {}

function deny(address usr) external auth {}

function file(bytes32 what, uint256 data) external auth {}
```

2.The `owner` role of the `PriceFeedAdapter` contract can replace the upstream price feed, modify `maxDelay`, change the bound asset, and pause or resume price reporting; this role directly affects the validity of collateral pricing and therefore impacts borrowing safety boundaries and liquidation behavior.

- rwausd-dss/src/adapter.sol#L62-L66, L69-L73, L75-L79, L81-L85, L88-L91, L93-L96

```
function transferOwnership(address newOwner) external onlyOwner {}

function setAsset(address newAsset) external onlyOwner {}

function setPriceFeed(address newFeed) external onlyOwner {}
```

```
function setMaxDelay(uint256 newDelay) external onlyOwner {}

function pause() external onlyOwner {}

function unpause() external onlyOwner {}
```

3. In the `rwaUSD` contract, `DEFAULT_ADMIN_ROLE` is the highest-level administrator, able to manage all privileged roles and resume contract operations; `UPGRADER_ROLE` is responsible for authorizing UUPS upgrades; `PAUSER_ROLE` can pause the contract; and `ccipAdmin` is used for CCIP token admin registry-related management.

- `rwaUSD-token-ccip/src/token/RWAUSD.sol#L214-L217, L220-L222, L228-L235, L243-L247, L251-L255`

```
function grantMintAndBurnRoles(address burnAndMinter) external {}

function getCCIPAdmin() external view returns (address) {}

function setCCIPAdmin(address newAdmin) external onlyRole(DEFAULT_ADMIN_ROLE) {}

function pause() public onlyRole(PAUSER_ROLE) {}

function unpause() public onlyRole(DEFAULT_ADMIN_ROLE) {}
```

Solution

In the short term, transferring owner ownership to multisig contracts is an effective solution to avoid single-point risk. But in the long run, it is a more reasonable solution to implement a privilege separation strategy and set up multiple privileged roles to manage each privileged function separately. And the authority involving user funds should be managed by the community, and the EOA address can manage the authority involving emergency contract suspension. This ensures both a quick response to threats and the safety of user funds.

Status

Acknowledged; Verification confirmed that the governance privileges across the protocol contracts have been transferred to the designated multisignature wallet.

[N11] [Suggestion] Missing zero address check

Category: Others**Content**

The `__RwaUsd_init` function lacks a zero address check for the `defaultUpgrader` parameter.

- `rwausd-token-ccip/src/token/RWAUSD.sol#L88-L102`

```
function __RwaUsd_init(
    string memory name,
    string memory symbol,
    uint8 decimals_,
    uint256 maxSupply_,
    uint256 preMint,
    address defaultAdmin,
    address defaultUpgrader
) internal onlyInitializing {
    __ERC20_init(name, symbol);
    __ERC20Burnable_init();
    __AccessControl_init();
    __AccessControlDefaultAdminRules_init(1 hours, defaultAdmin);
    __RwaUsd_init_unchained(decimals_, maxSupply_, preMint, defaultAdmin,
defaultUpgrader);
}
```

Solution

It is recommended to add a zero address check.

Status

Fixed

[N12] [Suggestion] Repeatedly triggered events**Category: Others****Content**

The corresponding event has been triggered internally in the Openzeppelin `PausableUpgradeable` contract, but the same event is being triggered repeatedly in the `pause` / `unpause` functions.

- rwausd-token-ccip/src/token/RWAUSD.sol#L243-L247, L251-L255

```
function pause() public onlyRole(PAUSER_ROLE) {
    _pause();

    emit Paused(msg.sender);
}

function unpause() public onlyRole(DEFAULT_ADMIN_ROLE) {
    _unpause();

    emit Unpaused(msg.sender);
}
```

Solution

The emit statement in the function is removed, which depends on OZ's internal implementation.

Status

Fixed

[N13] [Medium] ERC-7201 Storage slot namespace mismatch

Category: Others

Content

The annotation declares the namespace as `erc7201:chainlink.storage.rwaUSD`, while the inline comment in `RWAUSD_STORAGE_LOCATION`, which describes the calculation process, uses the string `multipli.storage.rwaUSD`. When performing upgrade security checks, @openzeppelin/upgrades-core parses the AST output by the Solidity compiler, extracts the namespace string from the `@custom:storage-location` annotation, and uses this as a basis to re-derive the expected slots, comparing it with the storage layout of the old and new implementations. If the namespace in the annotation does not match the slots actually used by the contract at runtime, the tool will complete the verification in an incorrect coordinate system. If the new implementation modifies the slots of critical variables such as `ccipAdmin`, `decimals`, and `maxSupply` during the upgrade, the detection tool may still silently allow the upgrade, leading to accidental overwriting of on-chain data.

- rwausd-token-ccip/src/token/RWAUSD.sol#L44, L55-L57

```
/// @custom:storage-location erc7201:chainlink.storage.rwaUSD

// keccak256(abi.encode(uint256(keccak256("multipli.storage.rwaUSD")) - 1)) &
~bytes32(uint256(0xff));

bytes32 private constant RWAUSD_STORAGE_LOCATION =
0xc1a8840385e35995fb7aabd6059552ccaa64bb804485e90cb444ac534e5ef600;
```

Solution

It is recommended to unify the namespace of the ERC-7201 storage slot.

Status

Fixed

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002603230001	SlowMist Security Team	2026.03.18 - 2026.03.23	Low Risk

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 2 medium risk, 2 low risk, 9 suggestion.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>